

一、基础配置与设置

1.1 预防降智配置

当 Claude Code 出现"降智"（表现变笨）时，可通过以下配置解决：

```
{
  "effortLevel": "high",
  "env": {
    "CLAUDE_CODE_DISABLE_1M_CONTEXT": "1",
    "CLAUDE_CODE_DISABLE_ADAPTIVE_THINKING": "1",
    "CLAUDE_CODE_DISABLE_AUTO_MEMORY": "1",
    "CLAUDE_CODE_SUBAGENT_MODEL": "sonnet"
  }
}
```

配置项	作用
<code>effortLevel: high</code>	设置思考强度为高
<code>CLAUDE_CODE_DISABLE_1M_CONTEXT</code>	禁用 1M 上下文缓存限制
<code>CLAUDE_CODE_DISABLE_ADAPTIVE_THINKING</code>	禁用自适应思维，提升稳定性
<code>CLAUDE_CODE_DISABLE_AUTO_MEMORY</code>	禁用自动记忆功能
<code>CLAUDE_CODE_SUBAGENT_MODEL</code>	子代理默认使用 sonnet 模型

1.2 完整 settings.json 配置

```
{
  "env": {
    "ANTHROPIC_AUTH_TOKEN": "your-token-here",
    "ANTHROPIC_BASE_URL": "https://openrouter.ai/api",
```

```

    "ANTHROPIC_DEFAULT_HAIKU_MODEL":
"openrouter/free",
    "ANTHROPIC_DEFAULT_OPUS_MODEL": "openrouter/free",
    "ANTHROPIC_DEFAULT_SONNET_MODEL":
"openrouter/free",
    "ANTHROPIC_MODEL": "openrouter/free",
    "ANTHROPIC_REASONING_MODEL": "openrouter/free",
    "CLAUDE_CODE_DISABLE_1M_CONTEXT": "1",
    "CLAUDE_CODE_DISABLE_ADAPTIVE_THINKING": "1",
    "CLAUDE_CODE_DISABLE_AUTO_MEMORY": "1",
    "CLAUDE_CODE_SUBAGENT_MODEL": "sonnet",
    "CLAUDE_CODE_NO_FLICKER": "1"
},
"model": "openrouter/free",
"skipDangerousModePermissionPrompt": true,
"permissions": {
    "allow": [
        "Bash(npm test *)",
        "Bash(npm run *)",
        "Bash(python *)",
        "Bash(node *)"
    ],
    "ask": [
        "Bash(git push *)",
        "Bash(git commit *)",
        "Bash(git merge *)"
    ],
    "deny": [
        "Bash(rm -rf *)",
        "Bash(format :)",
        "Bash(del *)"
    ]
}
}

```



完整配置包含更多设置，如需使用请参考 1.1 节预防降智配置中的关键环境变量。

1.3 NO_FLICKER 模式（无闪烁模式）

解决痛点：

- 彻底告别终端闪烁和内容跳动
- 支持鼠标操作（点击、移动光标）
- 长会话滚动位置更稳定，内存占用更友好

开启方式：

```
bash

# 临时开启
CLAUDE_CODE_NO_FLICKER=1 claude

# 永久生效 - 添加到 settings.json 的 env 中
"CLAUDE_CODE_NO_FLICKER": "1"
```



开启NO_FLICKER之后，你可以：[🔗 参照](#)

- 点击输入框定位光标，不用再按方向键一格一格移
- 点击折叠的工具输出，直接展开查看
- 点击 URL，打开浏览器
- 拖拽选中文本，自动复制到剪贴板

1.4 Auto Mode（自动模式）

核心优势：

- Claude 自己管理权限，不再每隔几秒就弹窗
- 适合跑复杂任务、测试、构建等长时间操作
- 内置安全检查机制

开启方式：

- 在 Claude Code 界面连续按 **3 次 Shift + Tab**



权限配置见 1.1 settings.json 中的 `permissions` 字段

二、Slash 命令与快捷操作

2.1 常用 Slash 命令

命令	功能	示例
<code>/init</code>	项目初始化，扫描代码库生成 CLAUDE.md	<code>/init</code>
<code>/focus</code>	开启禅模式，折叠工具细节	<code>/focus</code>
<code>/recap</code>	一键总结当前长会话	<code>/recap</code>
<code>/effort</code>	调整思考强度 (xhigh/high/max/ultrathink)	<code>/effort xhigh</code>
<code>/stats</code> / <code>/statusline</code>	查看 token 使用情况、状态栏	<code>/stats</code>
<code>/context</code>	查看上下文各部分占用 Token 情况	<code>/context</code>
<code>/usage</code>	查看当前额度剩余、剩余容量	<code>/usage</code>
<code>/vim</code>	开启 Vim 模式 (h j k l、ciw、dd 等)	<code>/vim</code>
<code>/rename</code>	给当前会话命名	<code>/rename api-migration</code>
<code>/sandbox</code>	配置沙盒模式，设置允许/禁止的操作范围	<code>/sandbox</code>
<code>/tools</code>	查看可用工具列表	<code>/tools</code>
<code>/skills</code>	查看已装载技能	<code>/skills</code>

命令	功能	示例
<code>/diff</code>	查看代码修改对比	<code>/diff</code>
<code>/compact</code>	压缩上下文，节省 Token	<code>/compact</code>
<code>/clear</code>	清空对话历史	<code>/clear</code>
<code>/btw</code>	插入不影响上下文的提问	<code>/btw 解释下 X</code>
<code>/model</code>	切换指定模型	<code>/model opusplan</code>
<code>/loop</code>	定时重复执行任务	<code>/loop 5m /test</code>
<code>/remote-control</code> / <code>/rc</code>	手机远程操控会话	<code>/rc</code>
<code>/plugin</code>	插件管理	<code>/plugin list</code>
<code>/doctor</code>	环境检测诊断	<code>/doctor</code>
<code>/branch</code>	创建分支会话	<code>/branch</code> <code>feature/foo</code>
<code>/memory</code>	管理持久记忆	<code>/memory list</code>
<code>/rewind</code>	回退对话	<code>/rewind 2</code>
<code>/export</code>	导出对话为 Markdown	<code>/export</code>
<code>/insights</code>	生成使用报告	<code>/insights</code>
<code>/simplify</code>	三维度并行代码审查	<code>/simplify</code>
<code>/buddy</code>	孵化/管理 AI 宠物	<code>/buddy feed</code>

2.2 常用快捷键

快捷键	功能
Ctrl + C	中断生成
Ctrl + L	清屏
Ctrl + J	换行
Ctrl + R	搜索历史
Alt + Enter	换行但不提交
Ctrl + U	删除整行输入
Ctrl + A	光标至行首
Ctrl + E	光标至行尾
Ctrl + G	打开长文本编辑器
Alt + P	快速切换模型
Ctrl + D	退出会话
Esc + Esc	快速中断当前操作但保留上下文
双击 Esc	对话回退、自动记忆整理
Ctrl + S	暂存当前提示词，稍后恢复
! 前缀	在提示中直接执行 Bash 命令，结果自动注入上下文



三、终端命令与操作

3.1 启动命令

```
bash

# 会话恢复
claude --continue          # 继续上次的对话
claude --resume            # 选择要恢复的会话
claude --resume <name>     # 按名字恢复指定会话

# 云端同步（从 claude.ai/code 拉取会话到本地）
claude --teleport <session_id>

# 跳过权限校验启动（Yolo 模式）
claude --dangerously-skip-permissions

# 非交互模式（适合脚本/自动化）
claude -p "简述这些更改"    # 直接输出结果
git diff | claude -p "Explain" # 管道输入

# 查看版本信息
claude -v
claude --version

# 更新客户端
claude update

# 查看 MCP 服务器
claude mcp servers list

# 查看全局 npm 包来源
npm list -g --depth=0

# 查看全部工具
claude tools list
```

3.2 提示内执行命令



bash

```
# 在提示中直接执行 shell 命令，结果自动塞进上下文
!git status
!npm test
!ls -la src/
```

四、AutoDream 与 AI 宠物

4.1 AutoDream 概览

开启步骤：

1. `claude -v` 检查版本（见 3.1 终端命令）
2. `/memory` 查看是否拥有 Auto Dream 灰度权限
3. 开启 Auto-dream 为 on 状态
' 关键指令：

- `/memory enable autodream` - 激活自动记忆整理



注： `/compact` 和 `/insights` 指令说明见第二章 Slash 命令表

注意： 切勿手动修改系统自动生成的记忆文件

4.2 AI 宠物配置

配置文件位置： `D:\project2026\zhishiku\02_Wiki\工具教程\Claude code.md`



json

```
{
  "companion": {
```



```
"name": "(宠物名字)",
"species": "(物种)",
"personality": "(性格描述)",
"rarity": "(稀有度)",
"stats": {
  "strength": 5,
  "agility": 7,
  "intelligence": 9,
  "charisma": 6,
  "luck": 4
}
}
```

常用指令：

- `/buddy` - 首次执行有孵化动画，生成专属 AI 宠物
- `/buddy feed` - 喂食

五、会话管理与控制技巧

5.1 核心技巧



注：以下快捷指令详细说明见第二章 Slash 命令表

- `/focus`：禅模式，配合 NO_FLICKER 使用最佳
- `/recap`：生成当前会话摘要
- `/loop`：定时执行任务

5.2 会话恢复

- `claude -c` - 恢复最近一次会话
- `/memory list` - 查看已保存的持久记忆

5.3 自动记忆整理

- 开启 AutoDream 后，会话结束自动生成记忆摘要
- 使用 `/compact` 在长会话前压缩上下文（见第二章 Slash 命令表）

六、相关资源

- [🔗 Claude Code 官方文档](#)
- [🔗 settings.json 配置参考](#)

七、@ 引用与上下文

7.1 @ Mentions 用法

给 AI 重点参考某个模块时，直接输入 `@` 引用：

语法	说明
<code>@src/auth.ts</code>	把单个文件引入上下文
<code>@src/components/</code>	参考整个目录
<code>@mcp:github</code>	启用或禁用 MCP 服务器

支持模糊匹配，能瞬间锁定目标文件或目录。

7.2 Memory Updates

直接告诉 Claude 更新项目记忆：



"更新 CLAUDE.md，记住这个项目用 bun 不用 npm。"

Claude 会自动把这条写入 CLAUDE.md 文件。

7.3 项目规则文件 (.claude/rules/)

创建 `.claude/rules/` 目录，放置不同模块的专门规则文件。

```
---
paths: src/api/**/*.ts
---
# API Development Rules
- All API endpoints must include input validation
```

- 每个 `.md` 文件自动加载为项目记忆
- 支持 YAML 前缀指定路径条件
- 只在处理匹配路径时生效

八、思考与规划模式

8.1 ultrathink（深度思考模式）

在 prompt 中加入 `ultrathink` 关键词，Claude 会分配最多 32K tokens 用于内部推理。

适合：架构设计、难缠 bug、性能优化
成本是普通对话的 3-5 倍，但输出质量明显更好。

8.2 Plan Mode（规划模式）

按两次 **Shift + Tab** 进入 Plan Mode。

Claude 会先分析项目代码和依赖，起草实现方案，等你批准后才开始写代码。

8.3 Extended Thinking (API)

调用 Claude API 时启用：

```
{
  "thinking": {
    "type": "enabled",
    "budget_tokens": 5000
  }
}
```

AI 会在 thinking blocks 里显示推理过程，便于调试复杂逻辑。

九、权限与安全

9.1 /sandbox 沙盒模式

提前设置允许的操作范围：

- 允许：`npm install`、`npm test`
- 禁止：`rm -rf`
- 支持通配符：`mcp__server__*` 允许所有 MCP 服务器
 - ！ 好处：不用每次都点允许，但危险操作自动拦截。

9.2 YOLO Mode

```
claude --dangerously-skip-permissions
```

跳过所有权限确认，适合隔离环境、可信操作。

9.3 Hooks 生命周期钩子

在 AI 执行流程中设置检查点：

事件	说明
PreToolUse	工具使用前
PostToolUse	工具使用后
PermissionRequest	权限请求时
Notification	通知发送时
SubagentStart/Stop	子代理启停

实际用途：

- 每次代码改动后自动运行测试
- push 前自动运行 lint
- 拦截危险命令（force push 到 main）
- 操作完成后发 Slack 通知

十、Subagents 与并行处理

10.1 Subagents

主 agent 分配任务，子 agent 并行执行。每个子 agent 独立拥有 200K context window。

子 agent 在后台运行，用 TaskOutput 获取结果。

就像一个项目经理带三个工程师，比单个全栈自己干更快。

10.2 Agent Skills

把公司的部署流程、测试方法论、文档规范打包成 Skill，团队其他人直接用。

10.3 Plugins

把 commands、agents、skills、hooks、MCP servers 全打包：



bash

```
/plugin install my-setup
```

一键配置整套 workflow。

十一、LSP 与代码理解

11.1 LSP Integration

Claude Code 支持 10 个 LSP 操作：

- 跳转定义、查找引用、悬停提示
 - 文档符号、工作区符号
 - 跳转实现、调用层级、被谁调用、调用了谁
- 这些操作让 AI 精准知道改这个函数会影响哪 37 个地方，不再靠猜。

十二、Chrome 浏览器集成

通过 Chrome 扩展，AI 可以直接操作浏览器。

能做的事： 导航页面、点击按钮、填表单、读控制台报错、截图

典型场景： 修复 bug 并在浏览器里验证

以前：你改代码 → 切浏览器 → 刷新 → 截图 → 发给 AI → AI 再改

现在：AI 自己完成整个循环

安装：claude.ai/chrome

十三、Claude Agent SDK

想把 Claude Code 能力集成到你自己的产品中：

```
javascript

import { query } from '@anthropic-ai/claude-agent-sdk';

for await (const msg of query({
  prompt: "Generate markdown API docs for all public functions in src/",
  options: {
    allowedTools: ["Read", "Write", "Glob"],
    permissionMode: "acceptEdits"
  }
})) {
  if (msg.type === 'result') console.log("Docs generated:", msg.result);
}
```

10 行代码就能让 AI 自动扫描代码、生成文档，也可用于代码审查、自动重构、测试生成。

十四、Boris Cherny 最佳实践

来自 Claude Code 创始人 Boris Cherny 的 69 条最佳实践（仓库 42k Star）：

15.1 别微操

贴上 bug 说 "fix"，Claude 自己会搞定。

尽量描述你想要实现的结果，而不是一步步指挥它怎么做。

15.2 及时压缩上下文

上下文超过 50% 时立刻使用 `/compact`，否则 AI 会进入"变蠢区间"。
使用 `/context` 可查看当前上下文占用情况。

15.3 CLAUDE.md 精简原则

- 不超过 200 行，多了 Claude 会忽略
- 包含：如何 build、如何 test、关键目录用途、代码规范
- 使用 `.claude/rules/` 目录存放各模块专门规则

十六、会话配置

16.1 会话保存周期

会话默认保存 30 天，可在配置中修改：

- `cleanupPeriodDays: 0` - 立刻清理
- `cleanupPeriodDays: 365` - 保留一年

16.2 自定义命令

把常用 prompt 保存为 `.md` 文件，自动变成 slash 命令：

```
/daily-standup      # 运行晨会流程
/explain $ARGUMENTS # 支持参数传递
```


十七、Windows 故障修复

17.1 常见问题

Windows 版 Claude Desktop 存在以下常见问题：

- 应用程序启动失败
- 陷入更新循环
- 在后台隐形运行

17.2 故障原因

1. 第三方平替工具（如 ClawGod）可能劫持了 `claude` 命令
2. 官方新版存在兼容性问题
3. 自动更新机制会将版本更新到有问题的版本

17.3 解决方案

步骤一：结束所有 Claude 进程



cmd

```
taskkill /F /IM claude.exe  
taskkill /F /IM "Claude Desktop.exe"
```

步骤二：卸载现有版本

通过 npm 安装：



cmd

```
npm uninstall -g @anthropic-ai/claude-code
```

通过 winget 安装：



cmd

```
winget uninstall Anthropic.ClaudeCode --source winget
```

步骤三：清理残留文件

删除以下目录（若存在）：

- `C:\Users\<用户名>\AppData\Local\AnthropicClaude`
- `C:\Users\<用户名>\.clawgod`

步骤四：安装旧版本

安装已知稳定的旧版本（如 v2.1.104）：

```
winget install Anthropic.ClaudeCode --source winget -v
2.1.104 --accept-source-agreements --accept-package-
agreements
```

步骤五：修复被劫持的命令

编辑 `C:\Users\<用户名>\.local\bin\claude.cmd`：

```
@echo off
"C:\Users\<用户名>
>\AppData\Local\Microsoft\WinGet\Packages\Anthropic.Cl
audeCode_Microsoft.Winget.Source_8wekyb3d8bbwe\claude.
exe" %*
```

步骤六：验证安装

```
claude --version
```

输出应为： `2.1.104 (Claude Code)`

17.4 预防措施

1. **阻止自动更新**：新版有问题时，不要运行 `winget upgrade`
2. **使用固定版本**：通过 `-v` 参数指定版本安装
3. **谨慎使用第三方平替**：可能会劫持命令，导致官方版无法使用

17.5 相关命令

命令	说明
<code>claude --version</code>	查看当前版本
<code>winget upgrade Anthropic.ClaudeCode</code>	检查更新
<code>winget install Anthropic.ClaudeCode -v x.x.x</code>	安装指定版本

十八、目录结构与文件说明

18.1 关键配置路径

文件	说明
<code>D:\project2026\zhishiku\02_Wiki\工具教程\Claude code.md</code>	当前文档路径
<code>C:\Users\<用户名>\.claude\settings.json</code>	核心全局设置文件
<code>C:\Users\<用户名>\.claude\config.json</code>	全局用户配置文件
<code>C:\Users\<用户名>\.claude\CLAUDE.md</code>	全局行为准则（所有项目生效）
<code>C:\Users\<用户名>\.claude\.claude.json</code>	项目级配置文件
<code>C:\Users\<用户名>\.claude\skill-manager-config.json</code>	技能管理器配置
<code>C:\Users\<用户名>\.claude\mcp_config.json</code>	MCP 服务器配置文件
<code>C:\Users\<用户名>\.claude\history.jsonl</code>	会话历史日志

18.2 projects 目录详解

`~/.claude/projects/` 是 Claude Code 的**项目级数据存储**，每个子文件夹对应你机器上的一个项目目录。

命名规则： 把项目路径中的 `\` 和 `:` 替换成 `--`

实际项目路径	对应文件夹名
<code>C:\Users\Administrator\Desktop</code>	<code>C--Users-Administrator-Desktop</code>
<code>D:\project2026\fuwari</code>	<code>D--project2026-fuwari</code>
<code>D:\project2026\zhishiku</code>	<code>D--project2026-zhishiku</code>

每个文件夹里存两类东西：

类型	文件格式	说明
会话记录	<code>uuid.jsonl</code>	每次聊天的完整对话记录，支持会话恢复
长期记忆	<code>memory/*.md</code>	跨会话保留的偏好、规范、踩坑记录等

目录结构示意：

```
~/.claude/projects/
├── C--Users-Administrator-Desktop/
│   ├── uuid1.jsonl      ← 会话记录
│   ├── uuid2.jsonl      ← 会话记录
│   └── memory/          ← 长期记忆
│       ├── MEMORY.md    ← 记忆索引
│       ├── user_preferences.md
│       └── wechat-article.md
├── D--project2026-fuwari/
│   ├── uuid3.jsonl
│   └── memory/
└── ...
```

关键点：

- 每个项目有**独立**的对话记录 + 记忆，互不干扰
- `memory/` 目录下的文件在新会话时自动加载
- 全局记忆放在 `~/.claude/CLAUDE.md`，所有项目生效

18.3 CLAUDE.md vs memory/MEMORY.md 的区别

两个文件都跟项目有关，但角色完全不同：

	项目根目录的 <code>CLAUDE.md</code>	<code>~/.claude/projects/xxx/memory/MEMORY.md</code>
放哪	项目根目录（跟着代码走）	Claude Code 内部目录（跟着工具走）
谁写	你手写或 <code>/init</code> 生成	Claude Code 自动生成
给谁看	给 Claude 看	给 Claude 记
作用	告诉 Claude 这个项目是什么、怎么构建、有哪些规范	记住 Claude 在做这个项目时学到的东西
类比	项目的 说明书	Claude 的 笔记本

打个比方：

- **CLAUDE.md** = 你给新入职员工的《项目手册》— 告诉他"这个项目是做什么的，目录结构怎么分，代码规范是什么"
- **memory/MEMORY.md** = 员工自己写的《工作笔记》— 记的是"上次改这里踩了个坑"老板不喜欢用 npm"

十九、学习渠道

- [🔗 Claude Code 官方快速入门](#)
- [🔗 Claude Code 精选资源列表](#)
- [🔗 菜鸟教程 Claude Code 教程](#)
- [🔗 Claude Code 最佳实践仓库](#)
- [🔗 Claude Code 学习笔记](#)

- [🔗 Claude Howto 学习路线](#)
- [🔗 高级设置 - Claude Code Docs](#)

相关来源

- [🔗 Anthropic 社区负责人连更31条Claude Code技巧！比Claude Code创始人私藏的还硬核](#)
- [🔗 预防降智设置](#)
- [🔗 Claude Code 推出 NO_FLICKER 模式](#)
- [🔗 Claude Code 新功能：自动模式。](#)
- [🔗 小互：Claude Code 推出：自动记忆功能](#)
- [🔗 https://claude.ai/](https://claude.ai/)
- [🔗 https://x.com/claudeai](https://x.com/claudeai)
- [🔗 飞书相关来源](#)
- [🔗 ClaudeCode远程安装 - 飞书云文档](#)
- [🔗 我安装的全部skill](#)